

Week Two Lecture Notes

Evan M Drake
drakeev@butte.edu

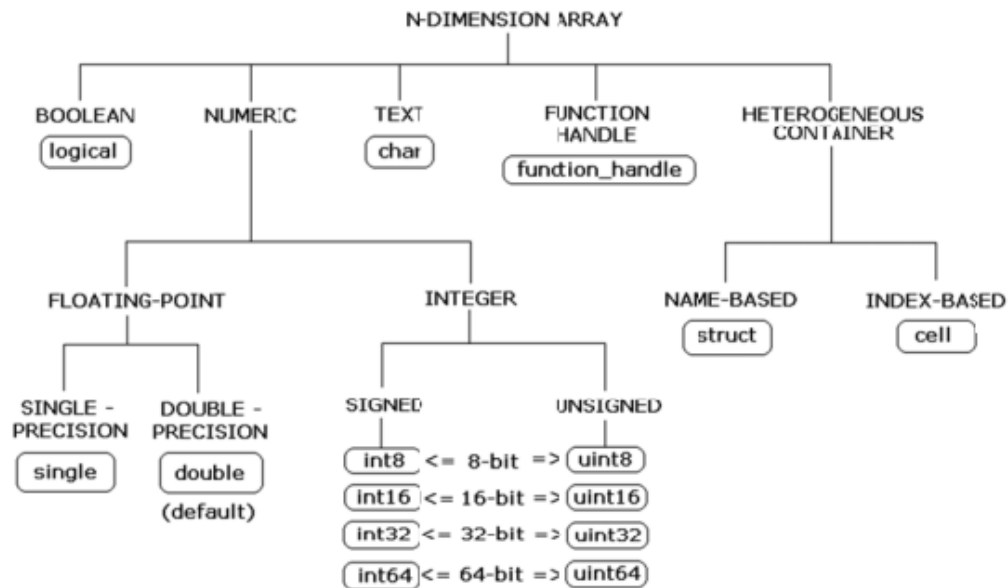
02/02/2025

Contents

1	Data Types in MATLAB	2
2	Common operators	2
2.1	Operator Precedence	3
3	Variables	3
3.1	Values / Expressions	4
3.2	The assignment operator	4
4	Collections	4
4.1	Array	4
4.2	Vector	4
4.3	Matrix	5
4.4	Collection operations	5
5	Linear Algebra	6
5.1	Recall	6
5.2	Reduced Row Echelon Form	7
5.3	Guass-Jordan Elimination	8
5.4	Matrix Algebra	10
	References	I
A	Helpful Links	II

1 Data Types in MATLAB

In the below we can see a graphic depicting the type tree of MATLAB. While giving you a good picture of how the types are related and what exactly they mean, this can look complicated. The idea to understand is that it is generally the case that specific data types such as numerics, often give rise to bodies of data types such as floating-point (real numbers), integers, complex numbers, which creates type hierarchies within languages that can be exploited when optimizing.



2 Common operators

Arithmetic is an important capability of MATLAB. As such the language contains all common arithmetic operators:

- Association: ()
- Unary: positive +, negative -
- Arithmetic: +, -, *, /, ^ log, \%
- Relational: <, >, <=, >=, ==, ~=

- Logical (Bitwise): `&`, `&&`, `|`, `||`

2.1 Operator Precedence

1. `()`
2. `^` `.''` (pure transpose)
3. `+` (unary plus) `-` (unary minus) `~` (NOT)
4. `*` `/` `\` `.*` `./` `./`
5. `+` (addition) `-` (subtraction)
6. `:`
7. `>` `<` `>=` `<=` `==` `~=`
8. `&` (AND)
9. `|` (OR)
10. `&&` (scalar AND)
11. `||` (scalar OR)
12. `=` (assignment)

3 Variables

A variable is a *value* bound to some arbitrary *name*. It is the case that the name must be some *word* (bound by the English alphabet and Arabic numerals in our case). Recall a we use word here as in set theory in which a word is some combination of values from a set. For example "and" is a word in English b/c $[a, n, d]$ are all from the set of English letters. Note that in MATLAB the names must start with a letter but can then contain arbitrary numbers.

3.1 Values / Expressions

Here a value is represented by any expression. An expression can be thought of as a kind of solvable, sequence of operations (equation), in discrete space¹. So an example of an expression is $f(x) = x + 5$ where $x + 5$ is our expression. This represents a solvable, sequence of operations that is seemingly in discrete space.

3.2 The assignment operator

The assignment operator `=` is how we will create variables in MATLAB. Note that in pseudocode they are often created using the binding operator `←`.

4 Collections

A collection can be thought of as simply a homogeneous "bag of stuff". Where everything in the bag is of the same type. We can think of a bag of marbles, each "thing" in the bag is a glass bead but maybe they are of varying sizes. We can extend the idea of collections to many different structures in type theory. But it is important to understand collections for pseudo code purposes as they can assist in optimization.

4.1 Array

An array is simply an ordered collection of finite, predetermined size. In particular this means that we must know the size of an array before we create it. An array has order and therefore each item in it has an index with which we can find it.

4.2 Vector

A vector is a bit different from an array in that it has no predetermined size. An array has dynamic size, in particular the size of a vector can be augmented on the fly for convenience. Vectors also have a concept of *direction*, this concept should be familiar from linear algebra. We can create row vectors and/or column vectors. Thus we can also create matrices.

¹we must always be in discrete space on a computer

4.3 Matrix

The bases of linear algebra is the matrix and thus the we will define it here. A matrix is such that it contains both row and column vectors, giving rise to the common visualization of a kind of box of squares. Note that a matrix can be given a flattened visualization that is simply a vector of vectors.

4.4 Collection operations

- Array Arithmetic: $+$, $-$, $*$, $/$ are mapped over the array space
- Vector arithmetic: inherits array arithmetic with extra rules for direction
- Matrix math: *transpose*, *multiplication* are the big ones here

Vector math can only be done between two vectors of the same length and direction. We wont worry about Cross multiplying as the calculation is easy to look up but difficult to memorize in our context.

$$\begin{aligned}\vec{u} + \vec{v} &\triangleq [u_1 + v_1, u_2 + v_2 \dots] \\ \vec{u} - \vec{v} &\triangleq [u_1 - v_1, u_2 - v_2 \dots] \\ \vec{u}\alpha &\triangleq [u_1\alpha, u_2\alpha \dots] \\ \vec{u} * \vec{v} &\triangleq \sum_i^n u_i v_i \text{s.t} \\ \text{Euclidean length } \vec{u} &\triangleq \sqrt{\sum_i^n u_i^2} \\ \text{where } n &\implies (|v| = |u|)\end{aligned}$$

The above are some of the common vector operations we will see in homework and labs. The most common Matrix operation in linear algebra is matrix multiplication as such we will deal with that often. Now I will urge you to revisit some linear algebra, I keep a reference with me and will probably have it in class often [2].

$$\begin{aligned}
A &= \mathbb{R}^{m \times l} \\
B &= \mathbb{R}^{l \times n} \\
|A * B| &\triangleq \mathbb{R}^{m \times n} \\
A * B &\triangleq A_R * B_C + A_{R+1} B_{C+1} + \dots
\end{aligned}$$

This will make much more sense after some muscle memory is used in the homework and assignments.

5 Linear Algebra

In the context of this class we use linear algebra to solve systems of equations. Nothing too horrible however some of the basics can be easily forgotten.

5.1 Recall

Recall that linear algebra is based on linear functions, of fns of the form $mx + b$ more formally. We as mathematicians utilize vectors and matrices to describe sets of linear equations (in this context). Recall that a vector can be thought of as a row or column that makes up some matrix informally. Matrices can be described in terms of their dimensions. In the context of this class we will only ever operate on 2D matrices, thus only two dimensions. We can write the dimensions of a *2D matrix* using the $\langle row \rangle \times \langle col \rangle$ notation.

$$M = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \quad (1)$$

Thus we can represent the matrix M from equation (1) as a 2×3 matrix. Notice that if we wanted to visualize rows it would be counting the horizontal lines. If we want to visualize the columns it is the vertical lines. Indexing a matrix is very similar in math.

$$A_{m,n} = \begin{bmatrix} a_{1,1} & a_{1,2} & \cdots & a_{1,n} \\ a_{2,1} & a_{2,2} & \cdots & a_{2,n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m,1} & a_{m,2} & \cdots & a_{m,n} \end{bmatrix} \quad (2)$$

We can see the notation for indexing a matrix in Eq. (2). We see clearly that the matrix A has its dimensions defined on as subscripts $A_{m,n}$. Recalling our *row*, *col* formula we can deduce that this means A is a matrix with m rows and n columns. From the matrix itself we can see that indexing individual elements is trivial using this formula. Notice that using this formula we can also define both column vectors $\vec{c}$ and row vectors $\vec{r}$.

$$\vec{c}_{a,1} = \begin{bmatrix} c_{1,1} \\ c_{2,1} \\ \vdots \\ c_{a,1} \end{bmatrix}$$

$$\vec{r}_{1,a} = [r_{1,1} \quad r_{1,2} \quad \cdots \quad r_{1,a}]$$

5.2 Reduced Row Echelon Form

Thus we can begin our refresher on linear algebra. We use linear algebra (in our context) to solve systems of equations.

$$\begin{aligned} 1x_1 + 2x_2 &= 5 \\ 3x_1 + 9x_2 &= 21 \end{aligned} \tag{3}$$

We can see from Eq (3) where we have a system of two equations. Reduced Row echelon form is a style of writing a matrix containing a system of equations. We first insert the equations into an *augmented matrix*.

$$\left[\begin{array}{cc|c} 1 & 2 & 5 \\ 3 & 9 & 21 \end{array} \right] \tag{4}$$

In Eq (5.2) we can see an augmented matrix of our system of equations. Note the vertical bar is only important for visualisation and this can be created as a matrix inside of matlab. We then use something like the Gauss-Jordan Elimination algorithm to create a Reduced Row Echelon Form (RREF) matrix.

$$\left[\begin{array}{cc|c} 1 & 0 & 1 \\ 0 & 1 & 2 \end{array} \right]$$

The RREF will reveal our result which here is the following.

$$x_1 = 1$$

$$x_2 = 2$$

We can also create this matrix by hand using the following rules [2]².

1. Add a multiple of one row to another row
2. Swap the position of two rows
3. Multiply a row by a constant

5.3 Guass-Jordan Elimination

The Guass-Jordan Elimination (GJE) method being an algorithm has specific rules defining a sequence of actions to create RREF. The algorithm proceeds in two phases: Forward (left to right) and Backward (right to left).

1. Forward
 - (a) Obtain a pivot (a leading one) in the leftmost column
 - (b) Subtract the row with the pivot from all rows below it to obtain zeros in the entire column.
 - (c) Look for a leading one in the next column and repeat.
2. Backward
 - (a) Find the rightmost pivot and use it to eliminate all numbers above the pivot in its column.
 - (b) Move one column other left and repeat.

We will work through an example to exemplify the *power* and *notation* of GJE now.

$$\begin{aligned} 1x + 2y + 3z &= 14 \\ 2x + 5y + 6z &= 30 \\ -1x + 2y + 3z &= 12 \end{aligned} \tag{5}$$

²This is almost never worth it use Guass-Jordan Elim unless otherwise advised.

Now we insert this system of equations into a matrix in Eq. (6)

$$\left[\begin{array}{ccc|c} 1 & 2 & 3 & 14 \\ 2 & 5 & 6 & 30 \\ -1 & 2 & 3 & 12 \end{array} \right] \quad (6)$$

Thus we can start GJE. In the first column there is already a one, if there were not a one a row swap would be necessary to proceed. Now following the GJE method we must make all indices below our *pivot* zero. We can accomplish this with the following operations.

$$\begin{aligned} R_2 &\leftarrow R_2 - 2R_1 \\ R_3 &\leftarrow R_3 + R_1 \end{aligned}$$

$$\left[\begin{array}{ccc|c} 1 & 2 & 3 & 14 \\ 0 & 1 & 0 & 2 \\ 0 & 4 & 6 & 26 \end{array} \right] \quad (7)$$

We have cleared the first column, now we need to find a pivot in the column. Again the leading one can be found, this time in row two. Using operation $R_3 \leftarrow R_3 - 4R_2$ we "clear" the column.

$$\left[\begin{array}{ccc|c} 1 & 2 & 3 & 14 \\ 0 & 1 & 0 & 2 \\ 0 & 0 & 6 & 18 \end{array} \right] \quad (8)$$

We now search for a pivot in the final column, finding a six instead. This means we will have to insert a one instead of happen upon it. We do this (in this case) using in place division: $R_3 \leftarrow \frac{1}{6}R_3$.

$$\left[\begin{array}{ccc|c} 1 & 2 & 3 & 14 \\ 0 & 1 & 0 & 2 \\ 0 & 0 & 1 & 3 \end{array} \right] \quad (9)$$

The forward phase is complete and the backwards phase begins. We must find the right most pivot and clear above it each time. In the third column we find our pivot. The following operation is required to clear its column: $R_1 \leftarrow R_1 - 3R_3$

$$\left[\begin{array}{ccc|c} 1 & 2 & 0 & 5 \\ 0 & 1 & 0 & 2 \\ 0 & 0 & 1 & 3 \end{array} \right] \quad (10)$$

The next pivot is in column two where again clearing its column is trivial. The operation being: $R_1 \leftarrow R_1 - 2R_2$ leaving the following matrix as the result in RREF.

$$\left[\begin{array}{ccc|c} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 2 \\ 0 & 0 & 1 & 3 \end{array} \right] \quad (11)$$

$\therefore x = 1, y = 2, z = 3.$

5.4 Matrix Algebra

Matrix algebra while not as useful computationally as GJE is still important to know about and understand. When we deal with multiple matrices (or column vectors) we can come into situations that look familiar. Take the equation $A\vec{x} = Z$ for instance, It is trivial to solve for any one of the variables above. In particular if we were to solve for $\vec{x}$ the obvious answer is $\vec{x} = \frac{Z}{A}$. This becomes more complex however when I say that A is a matrix.

$$\begin{bmatrix} 1 & 2 \\ 3 & 9 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 5 \\ 21 \end{bmatrix} \quad (12)$$

Where the above equation is equivalent to our original $A\vec{x} = Z$ but now A is a matrix and $\vec{x}, Z$ are vectors. Matrix division is not advisable so we invert $A \rightarrow A^{-1}$ and perform matrix multiplication. This allows for our original result of $\vec{x} = ZA^{-1}$. Matrix inversion by hand is out of the scope of this course.

[*]

References

- [1] Tom Rebold Nick Langhoff Erik Dunmire. *MATLAB*. 2017. URL: <https://canadacollege.edu/nsf-iuse/curriculum.php>.
- [2] Ivan Savov. *No Bullshit guide to linear algebra*. Minireference Co., 2020.

A Helpful Links

List of common commands